Received: 7 January 2020 | Revised: 5 October 2020 | Accepted: 20 October 2020

DOI: 10.1002/spe.2929

Check for updates

SURVEY PAPER

WILEY

Root causing, detecting, and fixing flaky tests: State of the art and future roadmap

Behrouz Zolfaghari[1,2] | **Reza M. Parizi**[3] | **Gautam Srivastava**[4,5] | **Yoseph Hailemariam**[3]

[1]Computer Science and Engineering Department, Indian Institute of Technology Guwahati, Guwahati, India

[2]Cyber Science Lab, School of Computer Science, University of Guelph, Ontario, Canada

[3]College of Computing and Software Engineering, Kennesaw State University, Marietta, Georgia,

[4]Department of Math and Computer Science, Brandon University, Brandon, Manitoba, Canada

[5]Research Center for Interneural Computing, China Medical University, Taichung, Taiwan

Correspondence
Gautam Srivastava, Department of Math and Computer Science, Brandon University, Brandon, MB, Canada.
Email: srivastavag@brandonu.ca

Abstract

A flaky test is a test that may lead to different results in different runs on a single code under test without any change in the test code. Test flakiness is a noxious phenomenon that slows down software deployment, and increases the expenditures in a broad spectrum of platforms such as software-defined networks and Internet of Things environments. Industrial institutes and labs have conducted a whole lot of research projects aiming at tackling this problem. Although this issue has been receiving more attention from academia in recent years, the academic research community is still behind the industry in this area. A systematic review and trend analysis on the existing approaches for detecting and root causing flaky tests can pave the way for future research on this topic. This can help academia keep pace with industrial advancements and even lead the research in this field. This article first presents a comprehensive review of recent achievements of the industry as well as academia regarding the detection and mitigation of flaky tests. In the next step, recent trends in this line of research are analyzed and a roadmap is established for future research.

KEYWORDS

detection, flaky testing, software, tools

1 | INTRODUCTION

In the recent few years, regression testing has been of interest to open source communities as it can identify code changes leading to new errors in previously tested codes after changes are committed in the repository by developers. Moreover, a regression test can ideally isolate the specific change in the code under test (CUT) or the test code that has affected the functionality (CUT is the code being tested, while test code refers to the code written for testing CUT). This is expected to facilitate and accelerate the code release process. But unfortunately, there are some code failures that are not caused by the latest change in the code, but by the flakiness of the test. A flaky test may nondeterministically pass or fail the same version of the CUT. Regression tests are costly themselves, and this usually causes developers to depend on regression test selection, which increases the rate of flaky tests.[1] Test flakiness may defeat the purpose of the regression test via slowing down software deployment. Flaky tests may adversely affect different test processes such as fault localization[2] and automated fix detection.[3]

Many big companies and teams are facing this problem. For example, in 2017, Google reported around 4.2 million tests that run on their continuous integration (CI) system. Among these tests, around 63 thousand exhibited at least one flaky

 © 2020 John Wiley & Sons, Ltd.

run over the course of a week (on average).[4,5] This makes less than 2% of their tests, it still causes significant drag on their engineers. Moreover, a report published in 2019 demonstrates that in Google CI testing, 84% of transitions from *"Pass"* to *"Fail"* are caused by flaky tests.[6] In the same year, a study on 211 projects in Apache Software Foundation showed that 21% of false alarms (false fail results) are created by flaky tests.[7] Another research work published in 2019 reports 5% flaky tests in Randoop (Obtained from five open-source Java projects with a total of 78 versions).[8] Moreover, Microsoft confirmed flaky tests to be the second most important reason (out of 10 reasons) adversely affecting software deployment time.[9] Moreover, it was reported in Reference 10 that 18% of test suite executions lead to failure and that 13% of these failures are caused by flaky tests. The authors of Reference 11 reported that 12% of tests run by developers from Java and C# IDEs show flaky behaviors. Furthermore, flaky tests were reported in separate studies on SAP HANA[12] and EvoSuite in Defects4j,[13] and Facebook is soliciting research proposals on flaky tests.[14] These statistics demonstrate that it is very important for software developers and testers to detect flaky tests and their sources quickly, precisely, and cost effectively.

The industrial software development community[15] as well as independent researchers[16,17] have started conducting research projects aiming at tackling the test flakiness problem. Many commercial software packages such as Azure[18] and Jenkins[19] have added plugins and facilities for detecting flaky tests. Moreover, a variety of tools[20-22] and datasets[23] have been developed to assist developers in detecting this kind of tests. Furthermore, attempts are made by well-known software teams and companies/organizations such as Google,[24] IEEE software,[21] and Apache[25] at educating developers on how to confront flaky tests. In addition, some theoretical approaches such as approaches based on machine learning (ML) have been presented by the industry for test flakiness detection purposes.[26]

Most of the methods suggested by industrial teams and institutes simply depend on test repetition. In these methods, a test is applied to the same CUT with the same test code for a given number of times. If the results are different, the test is deemed flaky.[27] This method is supported by a variety of commercial testing frameworks. To mention a few, one may refer to Android,[28] Jenkins,[29] Maven,[30] Spring,[31] and the Google TAP system.[32,33] But this approach is not reliable. Moreover, it engages a substantial amount of computing and storage resources and increases the time added to the developer feedback loop.[34]

In addition to the industrial community, the academic community has set out to conduct research flaky tests. Some universities such as Carnegie Mellon University,[35,36] University of Maryland,[37] and University of Texas[38] have held related seminars. Others are contributing via defining[39,40] and accomplishing[41] related theses. Some universities have started checking their own products and test frameworks for flakiness.[42] Moreover, academic researchers have started investigating different aspects of flaky tests such as their life cycle[43] and the related vocabulary.[44] However, the academic research community is behind the industry in this area.

A systematic and comprehensive review of the existing research works in this area can help to analyze the trends as well as establishing a future roadmap. This paves the way for further research and helps the academia keep pace with the industry and even lead the research on flaky tests. But to the best of our knowledge, there is not such a systematic review available in the literature. In this study, we take a systematic approach to the review of previous research works and tools aiming at the detection of flaky tests and their root causes. Moreover, we establish outlines for future research in this area. The main results obtained are: (1) there are four new methods used to detect flaky tests: code coverage counting, ML, test smells detection and order-dependency test; (2) code coverage counting is the most used method in most studies (33.3%) for the detection of flaky tests; (3) there are only two types of ways to fix flaky test; (4) there are 14 common causes for flakiness; (5) there are some key challenges for detecting flaky test. There needs to be more improvement in the detection of flaky tests and little research is available on the detection of the root causes of the flaky tests.

The rest of this article is organized as follows. Section 2 discusses the background and related works. In Section 3, we introduce the methodology used in our systematic review. Section 4 presents and analyzes the results of the review. Finally, in Section 5, we will conclude the article and suggest further research.

2 | BACKGROUND AND RELATED WORKS

This section gives brief background material to aid understanding of the article, and describe related works.

2.1 | Flaky test

The prevalence of flaky tests in software testing has reached new heights over the past few years. A flaky test is a test that exhibits both passing and failing results even though there is no code change in CUT or test code whose outcome is

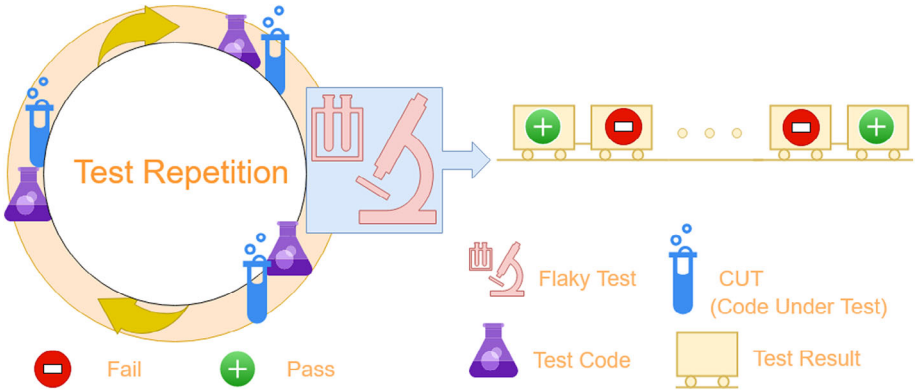


FIGURE 1 The schema of a flaky test process: The same CUT and the same test code can lead to different results in different runs. CUT, code under test [Color figure can be viewed at wileyonlinelibrary.com]

nondeterministic. In the industry, whenever developers write new code, it needs to be examined throughout the development process to make sure the software does what it is intended to do before releasing the software. In an ideal situation, when the test suite runs many times under the same CUT, it tells the engineer if the application is working or not. If it keeps on failing that would indicate that the code is buggy and it needs fixing, but if it passes all the rounds, it indicates that the code works. But sometimes, the outcome becomes nondeterministic; it would pass on some of the rounds and fail on other rounds. When the test fails to produce a consistent outcome, the test is called a flaky test.[45] Flaky text can occur from:

- New written code
- Test code
- Internal infrastructure such as operating system, hardware, and so forth.

Figure 1 shows a visual view of a flaky test. In this figure, the flaky test process is denoted by the arrow box. The big circle represents consecutive iterations of the test, and wagons labeled by green and red circles represent pass and fail outcomes, respectively. As shown in the figure, different iterations of the test lead to different outcomes. Flakiness substantially increases the test cost via hiding real bugs and wasting developers' time debugging false failures.[27] For example, consider the following two specifications.

Spec1:

```
module ApplicationHelper
 def logged\_in?
  Current.user.present?
 end
end
```

Spec2:

```
describe ApplicationHelper do
 describe "\#logged\_in¿' do
  it "returns false if the user is logged out" do
   expect(helper.logged\_in?).to be\_falsy
  end
  it "returns true if the user is logged in" do
   Current.user = create(:user)
   expect(helper.logged\_in?).to be\_truthy
  end
 end
end
```

The above two specs represent two CUTs trying to check if a user is logged in or not. In the test code, if *Spec1* and *Spec2* run in the order they have been defined, they will pass happily. But, if they are executed in the opposite order, then the statement *"returns false if the user is logged out"* will fail.

2.2 | Related works

To the best of our knowledge, there is no systematic survey on academic research works in the area of flaky tests. However, there are two research works somehow similar or relevant to our work in this article. As the first relevant work, we can refer to Reference 46. In the mentioned report, a survey is presented on 168 test smell sources. Test smells are a superset of flaky tests. From the sources studied in Reference 46, 120 are reported by the industry and 46 by academia. The authors of Reference 46 classified the sources carefully. The review presented in Reference 46 does not specifically focus on flaky tests while the statistics presented in Section 1 raise the need for independent research on flaky tests, like the one presented in this article.

The next relevant work is a multivocal literature review presented in Reference 47 that aims at highlighting the challenges and issues faced by developers when dealing with flaky tests. The authors of Reference 47 studied the developers' perspective when it comes to flaky tests. They asked developers to run 200 tests already known to be flaky. The goal was to find out how the developers try to fix the flakiness and which types of flaky tests are harder to fix. This research reports timeout problems as the most challenging ones for the developers to fix. Moreover, this research led to the identification of four new categories of causes for flaky tests; excessively restrictive range, test case timeout, platform dependency, and test suite timeout. The research reported in Reference 47 is clearly different from our systematic review, which studies all academic and industrial research works focusing on stopping, identifying, and resolving flaky tests.

The above discussions highlight the following shortcomings in existing surveys and reviews related to flaky tests, which raise the need for a systematic survey from an academic point of view as an unrolled stone.

- Some existing reviews such as the one presented in Reference 46 do not specifically focus on flaky tests. They discuss flaky tests as part of a whole. This leaves several aspects of test flakiness unaddressed.
- Some of them do not discuss the topic from an academic point of view. Reviews of this type cannot pave the way for future academic research on flaky tests.

3 | METHODOLOGY

In this article, we take a systematic approach to conduct the research. In this section, we will provide the procedures used to implement the systematic review. For this, we adopted well-established guidelines described in References 48 and 49.

Section 3.1 presents the goal and research questions for this article. Section 3.2 presents the search strategy and digital libraries used to get information about the studies. Section 3.3 describes the inclusion and exclusion criteria to filter the retrieved studies. Section 3.4 describes the procedures used to extract data from the articles. Finally, Section 3.5 gives an analytical perspective to the current trend of the research, presenting the publication trends.

3.1 | Goal and research questions

The goal of this study is (1) to show an overview of what has been done so far when it comes to flaky tests and (2) to identify promising research directions for further investigation. For this study, six research questions were defined, and each will be thoroughly investigated to get the best possible answers. Table 1 presents the questions we plan on answering after this systematic review.

3.2 | Search strategy

The most important word to search for this systematic literature review article was a flaky test and test flakiness. We used the keyword to search on IEEExplore, ACM, Scopus, Web of Science, and ScienceDirect databases because of their broad and in-depth studies in software engineering. Table 2 displays the search string for each database.

TABLE 1 Research questions

Research question	Motivational factor
RQ-1 What approaches and techniques have been used to detect flaky tests?	To review approaches and techniques commonly used for detecting flaky tests
RQ-2 What approaches and techniques have been used to fix flaky tests?	To review approaches and techniques commonly used for fixing flaky tests
RQ-3 What empirical studies have been done in test flakiness domain and overall quality of the studies?	Review of all the empirical studies in this domain
RQ-4 What are the root causes of test flakiness?	To identify the symptoms for flaky tests
RQ-5 What are the automated tool support and frameworks for detecting flaky tests?	To demonstrate the tools used in detecting flaky tests
RQ-6 What are the possible research directions based on the current issue/challenges in this domain?	To illustrate the current issues/challenges in solving flaky tests

TABLE 2 Search strings

Database	String
ACM, IEEExplore, ScienceDirect, CiteseerX, Scopus, Web of Science, John Wiley, Word Scientific	((“Flaky” OR “Flakiness OR “Intermittent OR “Nondeterministic) AND (“test” OR “testing”)) OR (“Test Flakiness”) OR (“Intermittent failures”)

3.3 | Inclusion and exclusion criteria

To ensure that the collected relevant works aligned with our systematic review, we defined several criteria to determine the works that would be considered and the ones that were outside the scope of this article. For a work to be considered for this systematic review, it must be a relevant work in the field of testing of software and systems and must be present in one of the online databases mentioned earlier. Refer to Table 3 to see the criteria.

3.4 | Study filtering

After we identified the online digital libraries to search for relevant works, we started the identification and collection of relevant works. We first identified what we called the “preliminary set of works.” To identify and collect the preliminary set of works, we performed string and manual keyword searches on the identified digital libraries. We applied the filtering process (a.k.a. pruning process) after applying search strings mentioned in Section 3.2 in order to discard the works which were unrelated to the goals of this research. The study filtering consists of four steps of pruning to the preliminary set of works to obtain the initial set of works for this review. In each step, we have applied the inclusion and exclusion criteria. The articles considered for this article did not have a restricted time frame to be considered a relevant study. Figure 2 shows the process of primary study selection. These pruning steps are briefly described below:

TABLE 3 Inclusion and exclusion criteria

Inclusion criteria	Exclusion criteria
IC-1: Books, articles, and technical reports describing studies regarding flaky test.	EC-1: Studies that lay outside the context of software testing
IC-2: Studies offer technical quality methods to combating flaky test	EC-2: Studies that are not published in English
IC-3: Studies that are available in technical archives or have a peer-review procedure	EC-3: Studies that do not report empirical findings and are only available in the form of short abstracts, Powerpoint presentation, white articles, editorial comments and book reviews

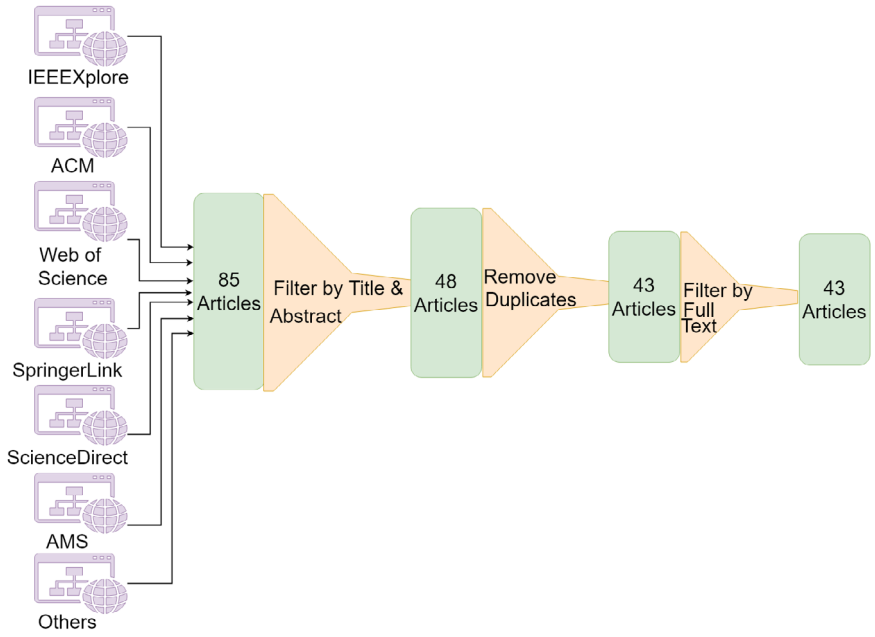


FIGURE 2 Study filtering process [Color figure can be viewed at wileyonlinelibrary.com]

Step 1: *Initial search* In this step, we applied the search string on the electronic databases. In total, 85 articles were collected in the initial search. All authors collaborated on this step by dividing the digital databases among themselves.

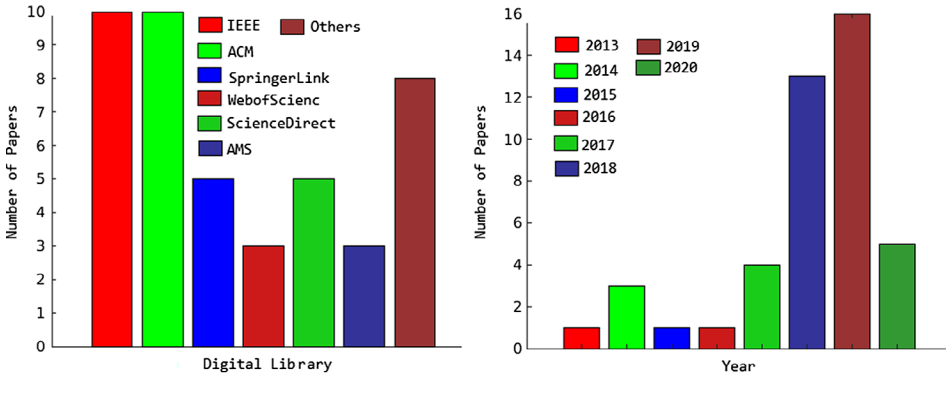
Step 2: *Filter by title and abstract* In this stage, we analyzed the articles considered based on their title and abstract. For this purpose, all exclusion criteria were applied. 48 articles were filtered after this step. Each author selected articles from the result of their search in the specific databases, assigned in Step 1. The selections were based on the titles and the abstracts. The outcome of this stage (from each author) was the set of full texts of the selected articles along with a report containing the titles and the abstracts of the rejected articles.

Step 3: *Remove duplicates* This step was taken by one author as the representative of the team of authors. There were some duplicate research articles from different databases, which were removed by the representative after collecting the individual reports from all authors.

Step 4: *Filter by full text* Just in case, a full-text analysis was implemented by the representative to make sure all the collected articles are relevant for this study. All authors agreed upon the final list of filtered articles after reading the report prepared by the representative. This report covered the titles of the articles along with the points and measures considered in the filtering process.

3.5 | Yearly and publication trends

Figure 3(A) shows the relevant studies found in each online database and Figure 3(B) shows the yearly trend of research in the flaky test domain.



(A) (B)

FIGURE 3 Publication trends and flaky test research [Color figure can be viewed at wileyonlinelibrary.com]

As it can be seen from the figure, the number of research reports focusing on flaky tests has increased from 1 in 2013 to 17 in 2019 (An obvious reason for the decrease in 2020 is that 2020 is not over in the moment of writing this report). The rapid growth in the number of research projects in this area is motivated by the prevalence of flaky tests identified and reported by the industry. Luo et al.[27] being the first ones to do flaky test research. Moreover, Figure 3highlights IEEE and ACM as the leading digital libraries when it comes to research on test flakiness.

4 | RESULTS AND ANALYSIS

In this section, we use the results obtained from reviewing relevant works to answer the research questions listed in Table 1. Furthermore, we comment on some insight after reviewing the relevant works on the future potential directions of research as well as limitations to the studies that have been performed already.

4.1 | What approaches and techniques have been used to detect flaky tests? (RQ-1)

In relevance to RQ-1 studies are proposing different techniques to detect flaky tests. The traditional way of detecting flaky tests, rerunning *N* number of times, consumes a substantial amount of resources, computing, and time. The approaches identified below are the proposed new and overarching techniques used by the research articles to detect flaky tests. The last two methods not only detect flaky tests but also detects the root causes of flaky tests. Some approaches have been proposed merely for some specific applications.[50] But in this subsection, we focus on general-purpose approaches.

In our study, we have observed research articles that utilize different *code coverage counting* methods. *Code coverage* is a measure of the number of lines in the software code that are run by a test. A test with a high code coverage improves the chance of discovering a software bug. Increasing the code coverage facilitates the detection of flaky tests via increasing the number of code lines covered by the test. Some research works have focused on improving code coverage in order to decrease the number of test runs required for detecting flaky tests. However, few research works have given provable results for decreasing the number of tests runs other than these techniques of reduction working well in the test cases as presented. In other words, some reported experimental results show a reduction in flakiness, but they are not supported by formal analysis or theoretical reasoning. Theoretically provable approaches to the reduction of flaky tests will be of notable value.

Flaky test detection is a computation-intensive process, which suggests ML as a suitable solution. ML gets computers to perform certain tasks without being explicitly programmed. Like other AI-based approaches, ML is now considered as an efficient approach to solving computation-intensive problems in different fields, for example, self-driving cars, practical speech recognition, email spam, malware filtering, and so forth. While other AI-based approaches have been frequently used, there are only a few research works that apply ad hoc ML-based approaches to the detection of flaky tests. Thus, rigorous research on the systematic application of ML in flakiness detection is strongly suggested.

In fact, considering the survey results garnered in References 27,34,47,51,52, we are envisioning that future works in this area should somehow depend on ML, deep learning (DL), or hybrid techniques.

Increase in *test smells* are one indicator for detecting flaky tests. Garousi and Küçük defined test smells as "poorly designed tests and their presence may negatively affect the quality of test suites and production code."[49[p1]]? According to some of the research articles, an increase in test smells leads to an increase of flaky tests. Therefore, different refactoring methods are used to fix poorly designed tests.

Order-dependent tests produce flaky tests due to the order of sequence in which the tests run. If there were test A and test B, changing the order of the tests on the CUT may result in different outcomes. As Luo et al.[27] had mentioned, the test-order dependency is one of the causes for flaky tests. We have observed two research articles utilize this type of method to detect flaky tests.

Finally, preliminary works have been done in *Instrumentation of tests and analyze logs of passing and failing runs*. This method helps in detecting the root causes of flaky tests. Only one study proposed this method and looks promising in this field. Table 4 gives a summary of the techniques for research articles that utilized the aforementioned methods.

Figure 4 shows the percentage of the different techniques used in detecting flaky tests and indicates that different types of detection techniques are being introduced since the inception of the flaky test. As shown in the figure, we found about 3 (33.3%) of the research articles used code coverage detection methods. It proves that code coverage is the most used method other than rerunning flaky tests. We believe that the reason why there is more research in code coverage

TABLE 4 Primary studies focusing on detecting flaky tests

Primary studies	Underlying technique	Inner working details	Limitations
53	Code coverage	Statement coverage	Coverage only shows you the code that you executed, not the code you verified
34	Code coverage	Differential coverage analysis	Coverage only shows you the code that you executed, not the code you verified
54	Code coverage	Statement coverage	Coverage only shows you the code that you executed, not the code you verified
55	Machine learning	Gradient boosted decision trees classifier	Deep learning not tackled
56	Machine learning	Bayesian network model	Somewhat dated methodology now
57	Increase in test smells	Resource optimism, indirect testing, and test run war	May not catch all issues of bad design
58	Increase in test smells	Resource optimism, indirect testing, test run war, fire and forget, and conditional test logic	May not catch all issues of bad design
59	Order-dependent test	Original order, random-class, random-class method, reverse class, and reverse-class method	Randomization will not lead to deterministic results
9	Instrumentation and analysis of log	Torch and root finder	Software dependent

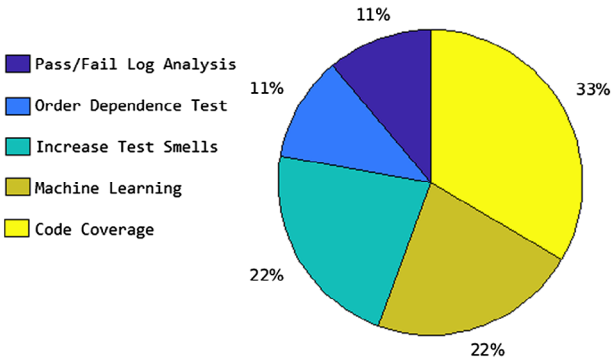


FIGURE 4 Detecting techniques percentage [Color figure can be viewed at wileyonlinelibrary.com]

is to make sure the test cases cover as many lines of code and reduce the number of reruns. Both ML and increase test smells accounts for 22.2% coming second. And finally, only one (11.1%) research used instrumentation and analysis of log files to detect flaky tests. Again here, we note that this is the current stage of these research distributions. Over time, we envision the percentage for ML techniques (Research) to overtake code coverage. We also envision Order Dependence and Pass/Fail Log analysis to diminish further in their slice of the pie.

4.2 | What approaches and techniques have been used to fix flaky tests? (RQ-2)

There are several approaches aiming at flakiness-aware[55] or flakiness-preventing test selection.[60] There are a few approaches to the prediction of flaky tests.[56] Some research works have focused on stopping flaky tests[61] or mitigating their effects.[53] Furthermore, some tools have been developed for the automatic detection of flaky tests[52] (discussed in the latter part of this article). Our results concerning this research question directed to the identification of below methods used in fixing flaky tests:

Refactoring of test order-dependent is one of the methods utilized to fix a flaky test where developers rearrange the test cases that have test-order dependency causes. The software will change the order of sequence in which the tests run using different arrangement settings. This technique helps to tackle one of the root causes of the flaky test. There has really been

very little research into the area of which types of refactoring to fix flaky methods may work best. Furthermore, should certain types of refactoring be avoided altogether would make for interesting areas for future work.

Configuration of testing tools are one of the methods used to fix flaky tests. Configuring, that is, concurrency lock, timeout strategies will prevent encounters with the flaky test. Unlike the *Refactoring of test order-dependent* described above, where it will fix the problem after it encounters flaky test, this method instead is prevention before testers encounter with flaky test. Users often keep unrealistic expectations from testing tools, and it takes a relatively long time for them to find out the real capabilities of the tools. Thus, long-term record and analysis is required in order to find out how they use the tools to detect flaky tests. In Table 5, we clear show the primary studies on fixing flaky tests.

Figure 5 illustrates the percentage of different ways to fix flaky tests. Configuration of testing tools accounts for 4 (80%) of the research articles that advised to fix flaky test and 20% used to refactoring test order dependency to fix flaky test. Throughout the study, we observed that detecting flaky tests has been of more interest to researchers than avoiding or resolving this kind of tests. In our opinion, the reason is that the detection of flaky tests is still a big hurdle in this domain.

4.3 | What empirical studies have been done in test flakiness domain and overall quality of the studies? (RQ-3)

Other than research on detecting and fixing flaky tests, five articles were primarily provided flaky-specific empirical studies. In Table 6, we have listed the empirical studies with description.

The empirical studies quality was assessed based on a set of ten criteria following the principles of a well-designed software engineering experiment listed in Reference 46. Table 7 shows the mentioned quality assessment criteria.

The criteria listed in Reference 46 were extracted after surveying 120 industrial and 46 academic sources. We were faced with some challenges while applying these criteria to the existing empirical studies. The main challenge was that these criteria are general and some of them were not applicable to some empirical studies. Table 8 presents the quality assessment results for the selected studies.

As it can be seen from Table 8, the majority of the empirical studies satisfy the set of criteria from the quality assessment. The articles have clearly stated their goals and delivered adequate information to support their objective. S02 and S16 have partially identified their experiment design, but we still believe good references for flaky tests.

TABLE 5 Primary studies focus on fixing flaky test

Primary studies	Underlying technique to fix flaky test	Common fixes
27,34,47,51	Configuration of testing tools	Configuring the Waitfor calls, sleep calls, adding locks, setting up/cleaning up states, removing dependency, and so forth
52	Refactoring of test order dependency	Minimizer and Patcher

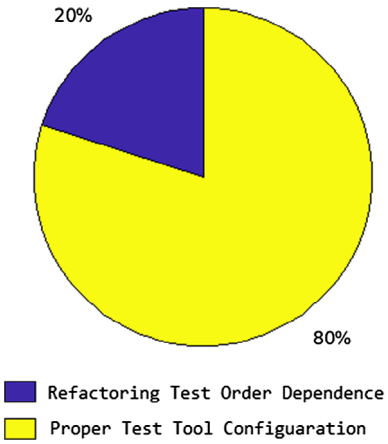


FIGURE 5 Fixing method percentage [Color figure can be viewed at wileyonlinelibrary.com]

TABLE 6 Empirical studies in test flakiness domain

Reference	Author	Description	Limitations
27	Luo et al.	Conducted the first extensive study of flaky tests by going over 201 commits from 51 apache projects and stated the root causes and strategies to fix flaky test.	Techniques are now somewhat dated (performed in 2014) and should be revisited
51	Throve et al.	Conducted a study to find flaky test in Android. From 29 GitHub Android projects, they found 77 commits that had flaky test and identify five root causes of the Android flaky test. They were also able to find three novel causes—dependency, program logic and UI, and many techniques are outdated for newer versions of Android (Lollipop and so forth). iOS and other mobile Operating systems were not tackled.	
47	Eck et al.	Conducted the first study to understand the developer's perspective. They analyzed 21 professional developers and surveyed 121 developers on how they viewed flaky tests based on nature, how they detected flaky test and fixed flaky tests. It is seen that flakiness is a big problem for the majority of the developers.	Very recent work. Very clearly described from the developer's viewpoint, however, lacked any future vision for what developers may face in the coming decade (2020 and on)
62	Rahman and Rigby	Studied the link between builds with failing tests and the number of crash reports on the Firefox browser. Since the developers usually ignores failing tests, it makes the test much more prone crash than if it is not ignoring the failing tests. In median circumstances, build with failing flaky test had 514 and 234 crashes for the Beta and Production channels, respectively. This study may be able to be extended to other browsers (Chrome, MS Edge). The study is now 2 years old so can be revisited by new researches with useful results.	
8	Paydar and Azamnouri	Studied Randoop, a well-known tool to generate test suites, on how flaky and fragile it is in regression test. The result shows that it has a low number of flaky accounting to only 5%.	One known limitation of Randoop is the inability to control values supplied as arguments to tests (which were randomized), making it impossible to guarantee equivalence partition coverage on inputs. This would make for interesting research to evaluate other test suites that do allow for such changes and/or a comparative analysis between test suites.

TABLE 7 Quality assessment criteria extracted from Reference 46

Identifier	Criteria	Description
QC1	Goal definition	Clear statement of the goals using goal/question/metrics
QC2	Context	Clear statement of context such as industrial or academic setting
QC3	Null hypothesis	Formal statement of the hypothesis
QC4	Variables	Identification of independent and dependent variables including, cost, effectiveness, and scalability measures
QC5	Experiment design	Selection of an appropriate standard design types such as one factor with two treatments
QC6	Comparison baseline	Selection of suitable benchmark(s)
QC7	Descriptive statistics	Appropriate reporting of descriptive statistics, such as measure of central tendency, dispersion and dependency, for selected cost, effectiveness, and scalability measures
QC8	Hypothesis testing	Decision about acceptance or rejection of hypothesis
QC9	Scalability analysis	Consideration of scalability issues
QC10	Validity	Consideration of possible risks associated with selected study such as internal, external, conclusion, and construct threats.

TABLE 8 The result of evaluation of quality

Study	QC1	QC2	QC3	QC4	QC5	QC6	QC7	QC8	QC9	QC10
S01	MC	MC	MC	MC	MC	MC	MC	MC	MC	MC
S02	MC	MC	MC	MC	PMC	MC	MC	MC	MC	MC
S03	MC	MC	MC	MC	MC	MC	MC	MC	MC	MC
S04	MC	MC	MC	MC	MC	MC	MC	MC	MC	MC
S16	MC	MC	MC	MC	PMC	PMC	MC	MC	MC	MC

Abbreviations: MC, met criteria; NMC, not met criteria; PMC, partial met criteria.

4.4 | What are the root causes of test flakiness? (RQ-4)

One of the earliest attempts at detecting the root causes of test flakiness was reported in Reference 27. This research highlights Asynchronous await, concurrency, and test order dependency issues as the main sources of test flakiness.

In addition to the general sources of flakiness, some domain-specific sources have been reported in the literature. For example, research was reported in Reference 51 that focuses on detecting flaky tests in Android. The authors of this article detected 77 flaky commits among 29 GitHub Android projects. Three new sources of flakiness were identified in this research; dependency, program logic, and UI. The issues relate to two of these items, namely, dependency[63] and UI[54] have been studied before, but not as sources of flakiness. Dependency has been reported as a source of test flakiness in some other research reports later.[64]

The link between builds with failing tests and the number of crash reports on the Firefox browser was studied in Reference 62. The authors of this article argued that developers usually ignore failing tests, and this makes the test more prone to crash. They reported 514 and 234 crashes for builds with failing flaky tests considering the Beta and Production channels, respectively in median circumstances.

In recent years, several research works have checked test procedures in a broad variety of platforms from Android cell phones[65] to large-scale industrial settings[9] in order to identify the root causes of flaky tests.

For this research question, we list all the root causes described by the primary studies. The causes are collected from different primary studies, mostly from empirical studies. Refer to Table 9 for the root causes of the flaky test.

Based on the primary studies, the common root causes for the flaky test are async wait and concurrency. Since both async wait and concurrency require the use of multiple computations (thread) happening at the same time, the order of execution might not follow the intended order and the outcome can be indeterminate. Async wait and concurrency can lead to indeterminate issues such as deadlocks, and resource starvations. There were some novice root causes discovered such as the program logic, test case timeout, and test suite timeout. The authors suggest future research on the root causes of flaky tests in state-of-the-art computing environments, platforms and paradigms such as Internet of Things, intelligent systems, cloud-based systems, and so forth.

4.5 | What are the automated tool support and frameworks for detecting flaky tests? (RQ-5)

We have observed that there is no standalone tool that focused on just detecting flaky tests, but there are many nonstandalone tools that cooperated functionalities to detect flaky tests. The majority of the tools use rerunning the test cases as the primary technique to detect flaky tests. In Table 10, we show currently used tools to detect flaky tests and their characteristics.

Table 10 suggests that there are different testing tools incorporating flaky test detection with different degrees of automation. Previously before the support from such testing tools, testers would have to devise their own rerunning function but now many testing tools are featuring flaky test detection functionality making the testing experience easier. But, from this systematic review, we have witnessed that there needs to be more work in this domain to develop much better automated support for flaky test detection. The lack of empirical evaluation of automated flaky testing tools in terms of effectiveness and efficiency is one of the areas of current limitations, as observed by the authors that could form a potential research avenue for fellow researchers.

TABLE 9 Root causes of test flakiness

Root causes of flaky test	Description	Reference
Async wait	It is a functionality that allows asynchronous, nonblocking call in synchronous setting. We identity Async wait when test execution does not properly wait for asynchronous calls.	27,42,47,57
Concurrency	Concurrency is when two task are being executed at the time. We can identify this type of flaky test, when we encounter different threads	27,47,51
Test-order dependency	It is caused due to the order of sequence in which the tests run	27,34,52-54
Resource leak	This occurs when the application do not properly allocate resources, for example, storage allocation, bad connections	27,47
Time	This occurs when different platform have different time settings. Nondeterministic failures happen if the test depends on the time.	27,47
Input/output	This occurs when the system does not properly receive the right input or return the right output	27
Randomness	It occurs when the test depends on random number generator	27,47
Floating point operation	Floating point calculation leads to nondeterministic behaviors, even simple calculations	27,47
Unordered collection	This occurs when the compiler assumes there is predefined order in Unordered collections.	27
Program logic	This occurs when developers fail to considered all corner cases to test.	51
User interface	When testing UI, developers may misinterpret the UI rendering process on the different devices which leads to nondeterministic failures	51
Platform dependency	Depending on the platform, nondeterministic can occurs. This is caused by the certain underlying operating system, certain hardware, or a third party tools.	27,47,51
Test case timeout	This occurs when a single test grows over time without reconfiguring the max runtime value.	47
Test suite timeout	This occurs when a test suite grows over time without reconfiguring the max runtime value.	47
Differential testing	To avoid the difficulties of fine-grained testing techniques such as unit tests, some developers and testers depend on differential testing, which is time-consuming and causes flaky tests.	66
Assumptions on deterministic implementations of nondeterministic specifications	Some commonly used test methods are based on nondeterministic specifications. For example, iterating through a set can return the elements in different orders. Deterministic implementations are often used for these nondeterministic specifications. This may lead to test flakiness.	67
Concurrency	Concurrent threads created by the test code(s) can cause nondeterminism in the execution order, and cause the test to exhibit flaky behaviors.	65
Test smells	–	57,58

4.6 | What are the possible research directions based on the current issues/challenges in this domain? (RQ-6)

In recent years, the literature of flaky tests has come with some useful conceptual and theoretical research projects such as establishing the life cycle of flaky tests[43] or standardizing the related vocabulary.[44] The review presented in this article takes the next step. Based on the results of this systematic literature review and our observations, we outline our roadmap for further research in this area as follows.

1. Formal modeling of flaky tests

   Formal modeling has been considered as one of the first steps in systematic research on many scientific topics[72] for past decades.[72] Especially, research in various branches of software engineering[73] and software testing[74] has been

TABLE 10 Flaky detection tools and their characteristics

Tools/framework presented	Detection technique	Distribution	Tool description
Selenium with TestNG[68]	Rerunning	Open-source	The tool has method retry to rerun the test N times if the tester wants. Another then rerun, selenium programmer use the waitFor command, which waits dynamically, checking for the desired condition every second
Maven Surefire[30]	Rerunning	Open-source	Tests uses rerunning until they pass or the number of reruns has been exhausted. This testing tool is mostly used in Java code.
Jenkins Flaky test handler[29]	Rerunning	Open-source	This plugin is used to provide various support for handling flaky tests and combination with maven, it is easy to do regression testing and aggregate and report flaky tests statistics. Using the function rerunFailingTestsCount from maven surefire, Jenkins lets users choose to rerun failed tests up to N times, and if it passes within any of those N times, the rerun will stop and the test will be marked as a "Flake." The build will be marked as successful if there are no failed tests but only "flaky" tests.
Pytest[69]	Rerunning	Open-source	This testing tool is used in python codes. The flaky plugin in the Pytest framework helps automatically reruns flaky tests and show a report of the results. Pytest also has a functionality to quarantine failed tests.
CircleCI[70]	Rerunning	Proprietary	CircleCI is leading cloud-native CI and it has a functionality to indicate which test is flaky or not
Android[71]	Rerunning	Open-source	The Android SDK provides testing functionalities. public abstract @interface FlakyTest handles flaky test detection by rerunning the test.
Spring	Rerunning	Open-source	It has a functionality called repeat that reruns test suites.
DeFlaker[34]	Code coverage	Open-source	This tool was created by one of the research article[34] and they used code coverage instead of rerunning.
IDFlakies[59]	Order dependent test	Open-source	This tool was devised from one of the research article[59] and uses changing of the test order to see if there is a flaky test.

started from formal modeling.[74] Moreover, the literature in the area of software testing has come up with several research works focusing on formal modeling in recent decades.[75,76] But, to the best of our knowledge, there is no formal modeling available for flaky tests. Thus, formal modeling, specification, and characterization of flaky tests along with their root causes and impacts is the very first research outline worthy of the investigation. This kind of modeling can help researchers predict the impacts of different unreported factors on test flakiness.

2. Presenting a taxonomy on flaky tests

   Taxonomies have been used to establish a sustainable line of research in many branches of software engineering,[77] and especially in software testing.[78] Based on the conducted study, there remains a need for a flaky test-specific taxonomy. Thus, future research may consider defining taxonomy of flaky tests in a clear context. This taxonomy should consider a variety of parameters such as causes, mechanisms, impacts, and so forth.

3. Developing standards for flakiness-free testing

   In recent years, several standards have been published to be used in software testing.[79-82] But, none of them covers test flakiness, as indicated by the results of this study. Therefore, the next avenue of future research on flaky tests is the development of standards for testing software with the minimum possibility of flakiness. These standards should specify the frameworks for test selection, test code development, and test oracle.

4. Investigation with AI-based approaches

   Different AI-based techniques such as ML and DL[83] as well as knowledge discovery in databases[84] are finding their applications in a broad variety of technological and scientific branches including software testing. It is pertinent to expect these techniques to be efficient in the detection of flaky tests as well. The current state of research in AI-assisted

approaches for flaky test classification is still infancy. As reported in this article, there are currently a few related research works[26,41] in this space, but this can be continued as a promising line of interest for fellow researchers.

5. Optimizing flaky test detection techniques

   The most used technique for combating flaky test, currently, is rerunning. Even though the primary studies listed in this article devised different approaches to detect flaky tests, the detection accuracy rate is relevantly similar or worse than rerunning flaky tests. Future research should lead to the development of more efficient and more accurate methods for detecting flaky tests. Such methods can help software development teams save substantial amounts of time and resources by eliminating the need for rerunning.

6. Integrating root cause detection studies

   There is not enough research on clearly classifying the root causes of flaky tests. The majority of the articles focus on detecting whether a test is flaky or not, but as important as that, there need to be tools to detect the root causes of flaky tests. Detection of root causes has not been adequately addressed by researchers while it has been shown in Reference 47 that on a scale of 0–3, the difficulty of identifying the root causes of test flakiness is 1.17 for software developers.

7. Automation support for fixing flaky tests

   As observed in this study, the majority of the literature deals with the detection aspect of flakiness. There are several tools for the automatic detection of flaky tests. However, there are very few methods of fixing aspects. Even worse, there is no automation support for fixing this kind of test. Thus, we suggest research works leading to the development of tools for the automatic elimination of flaky tests that help in effective debugging that could, in turn, increase the productivity of the developers.

8. Research on the flaky behaviors of tests in CI

   CI refers to the development practice wherein codes are integrated into a shared repository frequently each day. When a single test suite with the same settings is separately run in the context of local development and CI, the build may result in different outcomes wherein one setting it fails and on the other passes. As recognized in this study, there is no existing research performed to determine the reasons why such flakiness could happen in CI, particularly in Cloud-based environments. Thus, further research on the factors affecting this phenomenon would be necessary.

5 | CONCLUSION

Regression testing has become the standard for making sure if the software being built does what it intends to do. But lately, the presence of test flakiness undermines the trust of regression testing since the outcome is not deterministic. In this article, we highlighted flaky tests as an important class of problems increasing the time and cost of software code release and marketing. Then we showed that the industrial community is moving fast toward the development of methods and tools for detecting and fixing this kind of test. We argued that the research community has not moved as fast as the industry in this area. This gap motivated a systematic review of the existing relevant research works to analyze the state of the art and provides outlines for further research in this area. We comprehensively reviewed the literature and provided a future roadmap with possible areas that future research should focus on. Our work in this article can be continued by formally modeling flaky tests as well as their root causes ad impacts. Moreover, researchers can focus on developing flakiness-free standards for regression testing.

ORCID

Behrouz Zolfaghari https://orcid.org/0000-0001-6691-0988
Reza M. Parizi https://orcid.org/0000-0002-0049-4296
Gautam Srivastava https://orcid.org/0000-0001-9851-4103

REFERENCES

1. Shi A, Zhao P, Marinov D. Understanding and improving regression test selection in continuous integration. Paper presented at: Proceedings of the 2019 IEEE 30th International Symposium on Software Reliability Engineering; 2019; Berlin, Germany.
2. Vancsics B, Gergely T, Beszedes A. Simulating the effect of test flakiness on fault localization effectiveness. Paper presented at: Proceedings of the 2020 IEEE Workshop on Validation, Analysis and Evolution of Software Tests; 2020:London, ON, Canada.
3. Landsberg D, Barr E. Automated fix detection given flaky tests; 2018. arXiv:1810.02659 [cs.SE].

4. Google testing blog: where do our flaky tests come from? https://testing.googleblog.com/2017/04/where-do-our-flaky-tests-come-from.html. Accessed January 29, 2020.
5. Micco J. *The State of Continuous Integration Testing at Google*. Tokyo, Japan: Google; 2017.
6. The state of continuous integration testing @Google. https://static.googleusercontent.com/media/research.google.com/en//pubs/archive/45880.pdf. Accessed July 20, 2020.
7. Vahabzadeh A, Fard AM, Mesbah A. An empirical study of bugs in test code. Paper presented at: Proceedings of the 2015 IEEE International Conference on Software Maintenance and Evolution (ICSME); 2015; Bremen, Germany.
8. Paydar S, Azamnouri A. *An Experimental Study on Flakiness and Fragility of Randoop Regression Test Suites*. Paper presented at: In International Conference on Fundamentals of Software Engineering. Cham: Springer; 2019;111-126.
9. Lam W, Godefroid P, Nath S, Santhiar A, Thummalapenta S. Root causing flaky tests in a large-scale industrial setting. Paper presented at: Proceedings of the 28th ACM SIGSOFT International Symposium on Software Testing and Analysis; 2019; Beijing, China.
10. Labuschagne A, Holmes LIR. Measuring the Cost of Regression Testing in Practice: a study of Java projects using continuous integration. Paper presented at: Proceedings of the 2017 11th Joint Meeting on Foundations of Software Engineering; 2017; Paderborn, Germany.
11. Beller M, Gousios G, Panichella A, Proksch S, Amann S, Zaidman A. Developer testing in the IDE: patterns, beliefs, and behavior. *IEEE Trans Softw Eng*. 2019;45(3):261-284.
12. Bach T, Andrzejak A, Pannemans R. Coverage-based reduction of test execution time: lessons from a very large industrial project. Paper presented at: Proceedings of the 2017 IEEE International Conference on Software Testing, Verification and Validation Workshops (ICSTW); 2017; Tokyo, Japan.
13. Fan Z. A systematic evaluation of problematic tests generated by EvoSuite. Paper presented at: Proceedings of the 2019 IEEE/ACM 41st International Conference on Software Engineering: Companion Proceedings (ICSE-Companion); 2019; Montreal, QC, Canada.
14. Facebook testing and verification request for proposals; 2019. https://research.fb.com/programs/research-awards/proposals/facebook-testing-and-verification-request-for-proposals-2019/. Accessed July 20, 2020.
15. Flakiness dashboard HOWTO - the chromium projects. http://www.chromium.org/developers/testing/flakiness-dashboard. Accessed June 29, 2020.
16. Eradicating non-determinism in tests. https://martinfowler.com/articles/nonDeterminism.html. Accessed June 29, 2020.
17. Top 10 reasons for flaky automated tests. https://testguild.com/top-10-reasons-for-flaky-automated-tests/. Accessed July 20, 2020.
18. Manage flaky tests. https://docs.microsoft.com/en-us/azure/devops/pipelines/test/flaky-test-management?view=azure-devops. Accessed July 20, 2020.
19. Flaky test handler. https://plugins.jenkins.io/flaky-test-handler/; . Accessed July 20, 2020.
20. Mocha Flaky Test Finder. https://www.npmjs.com/package/mocha-flaky-test-finder; . Accessed July 20, 2020.
21. Efficiently and Automatically Detecting Flaky Tests with DeFlaker. http://blog.ieeesoftware.org/2018/04/efficiently-and-automatically-detecting.html; . Accessed July 20, 2020.
22. Test flakiness -- methods for identifying and dealing with flaky tests. https://engineering.atspotify.com/2019/11/18/test-flakiness-methods-for-identifying-and-dealing-with-flaky-tests/. Accessed July 20, 2020.
23. iDFlakies: flaky test dataset. https://sites.google.com/view/flakytestdataset. Accessed July 20, 2020.
24. Google testing blog: TotT: avoiding flakey tests. https://testing.googleblog.com/2008/04/tott-avoiding-flakey-tests.html. Accessed July 20, 2020.
25. Automated flaky test detector. https://cwiki.apache.org/confluence/display/MXNET/Automated+Flaky+Test+Detector. Accessed July 20, 2020.
26. A machine learning solution for detecting and mitigating flaky tests. https://medium.com/fitbit-tech-blog/a-machine-learning-solution-for-detecting-and-mitigating-flaky-tests-c5626ca7e853. Accessed July 20, 2020.
27. Luo Q, Hariri F, Eloussi L, Marinov D. An empirical analysis of flaky tests. Paper presented at: Proceedings of the 22nd ACM SIGSOFT International Symposium on Foundations of Software Engineering; 2014; Hong Kong, China.
28. FlakyTest | Android Developers. https://developer.android.com/reference/android/test/FlakyTest. Accessed June 29, 2020.
29. Flaky test handler plugin - Jenkins - Jenkins Wiki. https://wiki.jenkins.io/display/JENKINS/Flaky+Test+Handler+Plugin. Accessed June 29, 2020.
30. Maven surefire plugin – rerun failing tests. https://maven.apache.org/surefire/maven-surefire-plugin/examples/rerun-failing-tests.html. Accessed June 29, 2020.
31. Repeat (Spring Framework 5.2.0.RELEASE API). https://docs.spring.io/spring/docs/current/javadoc-api/org/springframework/test/annotation/Repeat.html. Accessed June 29, 2020.
32. Continuous integration at Google scale. https://eclipsecon.org/2013/sites/eclipsecon.org.2013/files/2013-03-24%20Continuous%20Integration%20at%20Google%20Scale.pdf. Accessed June 29, 2020.
33. Testing at the speed and scale of Google | Google engineering tools. http://google-engtools.blogspot.com/2011/06/testing-at-speed-and-scale-of-google.html. Accessed June 29, 2020.
34. Bell J, Legunsen O, Hilton M, Eloussi L, Yung T, Marinov D. *DeFlaker: Automatically Detecting Flaky Tests*. Paper presented at: In 2018 IEEE/ACM 40th International Conference on Software Engineering (ICSE), IEEE; 2018;433-444.
35. Seven recommendations for testing in a non-deterministic world. https://insights.sei.cmu.edu/sei_blog/2017/04/seven-recommendations-for-testing-in-a-non-deterministic-world.html. Accessed July, 20, 2020.
36. The challenges of testing in a non-deterministic world. https://insights.sei.cmu.edu/sei_blog/2017/01/the-challenges-of-testing-in-a-non-deterministic-world.html. Accessed July 20, 2020.

37. QUANTIFYING FLAKINESS AND MINIMIZING ITS EFFECTS ON SOFTWARE TESTING. https://drum.lib.umd.edu/handle/1903/20737. Accessed July 20, 2020.
38. Mitigating flaky tests. http://www.ece.utexas.edu/events/mitigating-flaky-tests. Accessed July 20, 2020.
39. PhD candidate in detecting (flaky) test failures of system user interactive tests (m/f). https://www.euraxess.fo/jobs/465178. Accessed July 20, 2020.
40. Ph.D candidate in detecting (flaky) test failures of system user interactive tests (m/f). https://scholarshipdb.net/scholarships-in-Luxembourg/Ph-D-Candidate-In-Detecting-Flaky-Test-Failures-Of-System-User-Interactive-Tests-M-F-University-Of-Luxembourg=UMVKuVQI6hGUWwAlkGUTnw.html. Accessed July 20, 2020.
41. Sjobom A. Studying Test Flakiness in Python Projects Original Findings for Machine Learning [Master's thesis]. School of Electrical Engineering and Computer Science, KTH Royal Institute of Technology in Stockholm. Stokholm, Sweden; 2019.
42. Presler-Marshall K, Horton E, Heckman S, Stolee K. Wait, wait. no, tell me. analyzing selenium configuration effects on test flakiness; 2019; Montreal, QC, Canada.
43. Lam W, Muslu K, Sajnani H, Thummalapenta S. A study on the life cycle of flaky tests. Proceedings of the ACM/IEEE 42nd International Conference on Software Engineering; 2020; (Virtually Held) Seoul, South Korea.
44. Pinto G, Miranda B, Dissanayake S, d'Amorim M, Treude C, Bertolino A. *What is the vocabulary of flaky tests?*. Paper presented at: Proceedings of the 17th International Conference on Mining Software Repositories; 2020:492-502.
45. What is a flaky test? definition from WhatIs.com. https://whatis.techtarget.com/definition/flaky-test. Accessed June 29, 2020.
46. Garousi V, Kucucb B. Smells in software test code: a survey of knowledge in industry and academia. *J Syst Softw*. 2018;138(1):52-81.
47. Eck M, Castelluccio FPM, Bacchelli A. Understanding flaky tests: the developer's perspective. Paper presented at: Proceedings of the 2019 27th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering; 2019; Tallinn, Estonia.
48. Petersen K, Feldt R, Mujtaba S, Mattsson M. Systematic mapping studies in software engineering. Paper presented at: Proceedings of the 12th International Conference on Evaluation and Assessment in Software Engineering (EASE); 2008; Swindon, UK.
49. Petersen K, Vakkalanka S, Kuzniarz L. Guidelines for conducting systematic mapping studies in software engineering: an update. *Inf Softw Technol*. 2015;64:1): 1-1): 18.
50. Dutta S, Shi A, Choudhary R, Zhang Z, Jain A, Misailovic S. Detecting flaky tests in probabilistic and machine learning applications; 2020.
51. Thorve S, Sreshtha C, Meng N. An empirical study of flaky tests in android apps. Paper presented at: Proceedings of the 2018 IEEE International Conference on Software Maintenance and Evolution; 2018; Madrid, Spain.
52. Shi A, Lam W, Oei R, Xie T, Marinov D. iFixFlakies: a framework for automatically fixing order-dependent flaky tests. Paper presented at: Proceedings of the 2019 27th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering; 2019; Tallinn, Estonia.
53. Shi A, Bell J, Marinov D. *Mitigating the effects of flaky tests on mutation testing*. Paper presented at: Proceedings of the 28th ACM SIGSOFT International Symposium on Software Testing and Analysis; 2019:112-122.
54. Memon AM, Cohen MB. Automated testing of GUI applications: models, tools, and controlling flakiness. Paper presented at: Proceedings of the 28th ACM SIGSOFT International Symposium on Software Testing and Analysis; 2013; San Francisco, CA.
55. Machalica M, Samylkin A, Porth M, Chandra S. Predictive test selection. Paper presented at: Proceedings of the 2019 IEEE/ACM 41st International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP); 2019; Montreal, QC, Canada.
56. King TM, Santiago D, Phillips J, Clarke PJ. Towards a Bayesian Network Model for Predicting Flaky Automated Tests. Paper presented at: 2018 IEEE International Conference on Software Quality, Reliability and Security Companion; 2018; Lisbon, Portugal.
57. Palomba F, Zaidman A. Notice of retraction: does refactoring of test smells induce fixing flaky tests? 2017 IEEE International Conference on Software Maintenance and Evolution; 2017; Shanghai, China.
58. Palomba F, Zaidman A. RETRACTION: the smell of fear: on the relation between test smells and flaky tests. *Empir Softw Eng*. 2019;24(1):2907-2946.
59. Lam W, Oei R, Shi A, Marinov D, Xie T. iDFlakies: a framework for detecting and partially classifying flaky tests. Proceedings of the 2019 12th IEEE Conference on Software Testing, Validation and Verification (ICST); 2019; Xi'an, China.
60. Leong C, Singh A, Papadakis M, Traon YL, Micco J. Assessing transition-based test selection algorithms at google 2019; Montreal, QC, Canada.
61. Mascheroni MA, Irrazábal E. Identifying key success factors in stopping flaky tests in automated REST service testing. *J Comput Sci Technol*. 2018;18(2):1-11.
62. Rahman MT, Rigby PC. The impact of failing, flaky, and high failure tests on the number of crash reports associated with Firefox builds; 2018; Lake Buena Vista, FL.
63. Zhang S, Jalali D, Wuttke J, et al. Empirically revisiting the test independence assumption. 2014; San Jose, CA.
64. Gambi A, Bell J, Zeller A. Practical test dependency detection; 2018; Vasteras, Sweden.
65. Dong Z, Tiwari A. Concurrency-related Flaky test detection in android apps; 2020. arXiv:2005.10762v2 [cs.SE].
66. Gulzar MA, Zhu Y, Han X. Perception and practices of differential testing; 2019; Montreal, QC, Canada.
67. Shi A, Gyori A, Legunsen O, Marinov D. Detecting assumptions on deterministic implementations of non-deterministic specifications; 2016; Chicago, IL.
68. Selenium and TestNG. https://testng.org/doc/selenium.html. Accessed July 20, 2020.
69. pytest: helps you write better programs. https://docs.pytest.org/en/latest/. Accessed July 20, 2020.
70. CircleCI: continuous integration and delivery. https://circleci.com/. Accessed July 20, 2020.

71. FlakyTest. https://developer.android.com/reference/androidx/test/filters/FlakyTest. Accessed July 20, 2020.
72. Das S, Karfa C, Biswas S. Formal modeling of network-on-chip using CFSM and its application in detecting deadlock. *IEEE Trans Very Large Scale Integrat (VLSI) Syst*. 2020;28(14):1016-1029.
73. Cremers A, Hibbard T. Formal modeling of virtual machines. *IEEE Trans Softw Eng*. 1978;SE-4(5):426-436.
74. Podgurski A, Clarke LA. Dependences and its implications for software testing, debugging, and maintenance. *IEEE Trans Softw Eng*. 1990;16(9):965-979.
75. Cangussu J, DeCarlo R, Mathur A. A formal model of the software test process. *IEEE Trans Softw Eng*. 2002;28(8):782-796.
76. Buzhinsky I, Pang C, Vyatkin V. Formal modeling of testing software for cyber-physical automation systems. Paper presented at: Proceedings of the 2015 IEEE Trustcom/BigDataSE/ISPA; 2015; Helsinki, Finland.
77. Shameem M, Khan AA, Hasan MG, Akbar MA. Analytic hierarchy process based prioritisation and taxonomy of success factors for scaling agile methods in global software development. *IET Softw*. 2020;14(4):389-401.
78. Grossmann J, Felderer M, Viehmann J, Schieferdecker I. A taxonomy to assess and tailor risk-based testing in recent testing standards. *IEEE Softw*. 2020;37(1):40-49.
79. ISO Central Secretary ISO/IEC/IEEE draft international standard - software and systems engineering –software testing–Part 1: concepts and definitions. standard ISO/IEC TR P29119-1/CD, International Organization for Standardization; 2020.
80. ISO Central Secretary ISO/IEC/IEEE draft international standard - software and systems engineering - software testing – Part 2: Test processes. Standard ISO/IEC TR P29119–2/DIS Second edition, International Organization for Standardization; 2020.
81. ISO Central Secretary ISO/IEC/IEEE Draft International Standard - Software and systems engineering – software testing –Part 3: Test documentation. Standard ISO/IEC TR P29119-3/DIS Second edition, International Organization for Standardization; 2020.
82. ISO Central Secretary IEEE/ISO/IEC Draft International Standard - Software and systems engineering–Software testing–Part 4: Test techniques. Standard ISO/IEC TR P29119-4/DIS, International Organization for Standardization; 2020.
83. Durelli VHS, Durelli RS, Borges SS, et al. Machine learning applied to software testing: a systematic mapping study. *IEEE Trans Reliab*. 2019;68(3):1189-1212.
84. Wang H, Bai L, Jiezhang M, Zhang J, Li Q. Software testing data analysis based on data mining. Paper presented at: Proceedings of the 2017 4th International Conference on Information Science and Control Engineering (ICISCE); 2017; Changsha, China.

How to cite this article: Zolfaghari B, Parizi RM, Srivastava G, Hailemariam Y. Root causing, detecting, and fixing flaky tests: State of the art and future roadmap. *Softw Pract Exper*. 2020;1–17. https://doi.org/10.1002/spe.2929